

OOP Design Patterns in Java

A Comprehensive Reference with Examples

March 2026

1. Introduction

Design patterns are reusable, proven solutions to recurring problems in software design. First formalised by the "Gang of Four" (GoF) in their landmark 1994 book *Design Patterns: Elements of Reusable Object-Oriented Software*, they represent the collective wisdom of experienced developers across decades of practice.

Patterns are not finished code — they are templates or blueprints that you adapt to your specific context. Knowing when to apply (and when to avoid) a pattern is just as important as knowing how to implement it.

The Three Categories

- Creational — control how objects are created
- Structural — define how objects and classes are composed
- Behavioural — manage algorithms and responsibilities between objects

2. Creational Patterns

Creational patterns abstract the instantiation process, making a system independent of how its objects are created, composed, and represented.

2.1 Singleton

Attribute	Detail
Intent	Ensure a class has only one instance and provide a global access point.
When to use	Shared resources: logging, configuration, thread pools, caches.
Trade-offs	Hides dependencies; harder to unit-test; can introduce tight coupling.

Java Example

```
public class AppConfig {  
    // volatile ensures visibility across threads  
    private static volatile AppConfig instance;  
    private String dbUrl;  
    private AppConfig() {  
        this.dbUrl = "jdbc:mysql://localhost:3306/mydb";  
    }  
    // Double-checked locking (thread-safe, lazy initialisation)
```

```

public static AppConfig getInstance() {
    if (instance == null) {
        synchronized (AppConfig.class) {
            if (instance == null) {
                instance = new AppConfig();
            }
        }
    }
    return instance;
}

public String getDbUrl() { return dbUrl; }
}

// Usage
AppConfig config = AppConfig.getInstance();
System.out.println(config.getDbUrl());

```

2.2 Factory Method

Attribute	Detail
Intent	Define an interface for creating an object, but let subclasses decide which class to instantiate.
When to use	When the exact type of object to create isn't known until runtime.
Trade-offs	Can lead to a large number of subclasses if overused.

Java Example

```

// Product interface
interface Notification {
    void send(String message);
}

// Concrete products
class EmailNotification implements Notification {
    public void send(String message) {
        System.out.println("Email: " + message);
    }
}

```

```

    }
    class SmsNotification implements Notification {
    public void send(String message) {
    System.out.println("SMS: " + message);
    }
    }
    // Creator
    abstract class NotificationService {
    // Factory Method
    public abstract Notification createNotification();
    public void notify(String msg) {
    Notification n = createNotification();
    n.send(msg);
    }
    }
    class EmailService extends NotificationService {
    public Notification createNotification() {
    return new EmailNotification();
    }
    }
    // Usage
    NotificationService svc = new EmailService();
    svc.notify("Your order has shipped!");

```

2.3 Builder

Attribute	Detail
Intent	Separate the construction of a complex object from its representation.
When to use	Objects with many optional parameters; avoids telescoping constructors.
Trade-offs	More code; overkill for simple objects.

Java Example

```

public class HttpRequest {
    private final String url;
    private final String method;
    private final String body;
    private final int timeout;
    private HttpRequest(Builder b) {
        this.url = b.url;
        this.method = b.method;
        this.body = b.body;
        this.timeout = b.timeout;
    }
    public static class Builder {
        private final String url;
        private String method = "GET";
        private String body = "";
        private int timeout = 30;
        public Builder(String url) { this.url = url; }
        public Builder method(String m) { this.method = m; return this; }
        public Builder body(String b) { this.body = b; return this; }
        public Builder timeout(int t) { this.timeout = t; return this; }
        public HttpRequest build() { return new HttpRequest(this); }
    }
}

// Usage — fluent API
HttpRequest req = new HttpRequest.Builder("https://api.example.com/data")
    .method("POST")
    .body("{\"key\":\"value\"}")
    .timeout(60)
    .build();

```

2.4 Prototype

Attribute	Detail
Intent	Create new objects by copying (cloning) an existing object.
When to use	Object creation is expensive; you need many similar objects quickly.
Trade-offs	Deep vs shallow copy must be managed carefully.

Java Example

```

public class Shape implements Cloneable {
    private String type;
    private int x, y;
    public Shape(String type, int x, int y) {
        this.type = type; this.x = x; this.y = y;
    }
    @Override
    public Shape clone() {
        try {
            return (Shape) super.clone(); // shallow copy
        } catch (CloneNotSupportedException e) {
            throw new RuntimeException(e);
        }
    }
    @Override
    public String toString() {
        return type + " at (" + x + ", " + y + ")";
    }
}

// Usage
Shape original = new Shape("Circle", 10, 20);
Shape copy = original.clone();
System.out.println(copy); // Circle at (10,20)

```

3. Structural Patterns

Structural patterns deal with object composition, creating relationships between objects to form larger structures.

3.1 Adapter

Attribute	Detail
Intent	Convert the interface of a class into another interface that clients expect.
When to use	Integrating legacy or third-party code with an incompatible interface.
Trade-offs	Extra indirection layer; can obscure the original contract.

Java Example

```
// Existing (legacy) class
class LegacyPrinter {
    public void printDocument(String text) {
        System.out.println("[Legacy] " + text);
    }
}

// Target interface expected by client code
interface ModernPrinter {
    void print(String text);
}

// Adapter wraps legacy and exposes modern interface
class PrinterAdapter implements ModernPrinter {
    private final LegacyPrinter legacy;

    public PrinterAdapter(LegacyPrinter legacy) {
        this.legacy = legacy;
    }

    @Override
    public void print(String text) {
        legacy.printDocument(text);
    }
}

// Usage
ModernPrinter printer = new PrinterAdapter(new LegacyPrinter());
```

```
printer.print("Hello, World!");
```

3.2 Decorator

Attribute	Detail
Intent	Attach additional responsibilities to an object dynamically without subclassing.
When to use	When extension by subclassing would lead to a class explosion.
Trade-offs	Many small objects; order of decorators can matter.

Java Example

```
interface Coffee {
    String getDescription();
    double getCost();
}

class SimpleCoffee implements Coffee {
    public String getDescription() { return "Coffee"; }
    public double getCost() { return 1.00; }
}

// Base decorator
abstract class CoffeeDecorator implements Coffee {
    protected final Coffee coffee;
    CoffeeDecorator(Coffee c) { this.coffee = c; }
}

class MilkDecorator extends CoffeeDecorator {
    MilkDecorator(Coffee c) { super(c); }
    public String getDescription() { return coffee.getDescription() + ", Milk"; }
    public double getCost() { return coffee.getCost() + 0.25; }
}

class SugarDecorator extends CoffeeDecorator {
    SugarDecorator(Coffee c) { super(c); }
    public String getDescription() { return coffee.getDescription() + ", Sugar"; }
    public double getCost() { return coffee.getCost() + 0.10; }
```

```

    }
    // Usage
    Coffee c = new SugarDecorator(new MilkDecorator(new SimpleCoffee()));
    System.out.println(c.getDescription() + " $" + c.getCost());
    // Coffee, Milk, Sugar $1.35

```

3.3 Facade

Attribute	Detail
Intent	Provide a simplified interface to a complex subsystem.
When to use	You want to provide a simple interface to complex subsystem logic.
Trade-offs	Can become a "god object" if not kept focused.

Java Example

```

// Complex subsystems
class CPU { public void start() { System.out.println("CPU started"); } }

class Memory { public void load() { System.out.println("Memory loaded"); } }

class Disk { public void read() { System.out.println("Disk read"); } }

// Facade
class ComputerFacade {
    private final CPU cpu = new CPU();
    private final Memory memory = new Memory();
    private final Disk disk = new Disk();
    public void startComputer() {
        cpu.start();
        memory.load();
        disk.read();
        System.out.println("Computer is ready.");
    }
}

// Usage — client only knows Facade

```



```
new ComputerFacade().startComputer();
```

3.4 Composite

Attribute	Detail
Intent	Compose objects into tree structures to represent part-whole hierarchies.
When to use	File systems, menus, UI component trees — anything with nested groups.
Trade-offs	Can make design overly general; type safety may need care.

Java Example

```
import java.util.*;

interface FileSystemItem {
    void display(String indent);
}

class File implements FileSystemItem {
    private final String name;

    File(String name) { this.name = name; }

    public void display(String indent) {
        System.out.println(indent + "- " + name);
    }
}

class Folder implements FileSystemItem {
    private final String name;
    private final List<FileSystemItem> items = new ArrayList<>();

    Folder(String name) { this.name = name; }

    public void add(FileSystemItem item) { items.add(item); }

    public void display(String indent) {
        System.out.println(indent + "+ " + name);
        for (FileSystemItem i : items) i.display(indent + " ");
    }
}

// Usage
Folder root = new Folder("root");
```

```

root.add(new File("readme.txt"));
Folder src = new Folder("src");
src.add(new File("Main.java"));
root.add(src);
root.display("");

```

4. Behavioural Patterns

Behavioural patterns are concerned with algorithms and the assignment of responsibilities between objects.

4.1 Observer

Attribute	Detail
Intent	Define a one-to-many dependency so that when one object changes state, all dependents are notified.
When to use	Event systems, MVC frameworks, real-time dashboards.
Trade-offs	Unexpected updates; hard to track cascading changes in large systems.

Java Example

```

import java.util.*;

interface Observer {
    void update(String event);
}

class EventBus {
    private final List<Observer> observers = new ArrayList<>();
    public void subscribe(Observer o) { observers.add(o); }
    public void unsubscribe(Observer o) { observers.remove(o); }
    public void publish(String event) {
        for (Observer o : observers) o.update(event);
    }
}

// Usage
EventBus bus = new EventBus();
bus.subscribe(event -> System.out.println("Logger: " + event));
bus.subscribe(event -> System.out.println("Analytics: " + event));
bus.publish("user.signup");

```

4.2 Strategy

Attribute	Detail
Intent	Define a family of algorithms, encapsulate each one, and make them interchangeable.
When to use	When you want to switch algorithms at runtime (sorting, pricing, routing).
Trade-offs	Clients must be aware of different strategies.

Java Example

```
// Strategy interface
interface SortStrategy {
    void sort(int[] data);
}

class BubbleSort implements SortStrategy {
    public void sort(int[] data) {
        // bubble sort logic
        System.out.println("Bubble sort applied");
    }
}

class QuickSort implements SortStrategy {
    public void sort(int[] data) {
        // quick sort logic
        System.out.println("Quick sort applied");
    }
}

// Context
class Sorter {
    private SortStrategy strategy;

    public void setStrategy(SortStrategy s) { this.strategy = s; }
    public void sort(int[] data) { strategy.sort(data); }
}

// Usage
Sorter sorter = new Sorter();
```

```

    sorter.setStrategy(new QuickSort());

    sorter.sort(new int[] {5, 3, 1, 4, 2});

```

4.3 Command

Attribute	Detail
Intent	Encapsulate a request as an object, allowing undo/redo, queuing, and logging.
When to use	GUI actions, transaction systems, macro recording.
Trade-offs	Proliferation of small command classes.

Java Example

```

interface Command {
    void execute();
    void undo();
}

class TextEditor {
    private final StringBuilder text = new StringBuilder();

    public void append(String s) { text.append(s); }

    public void deleteLast(int len) { text.delete(text.length() - len,
        text.length()); }

    public String getText() { return text.toString(); }
}

class TypeCommand implements Command {
    private final TextEditor editor;
    private final String typed;

    TypeCommand(TextEditor e, String t) { editor = e; typed = t; }

    public void execute() { editor.append(typed); }

    public void undo() { editor.deleteLast(typed.length()); }
}

// Usage
TextEditor editor = new TextEditor();
Command cmd1 = new TypeCommand(editor, "Hello");
Command cmd2 = new TypeCommand(editor, " World");
cmd1.execute(); cmd2.execute();

```

```

System.out.println(editor.getText()); // Hello World

cmd2.undo();

System.out.println(editor.getText()); // Hello

```

4.4 Template Method

Attribute	Detail
Intent	Define the skeleton of an algorithm in a base class, deferring some steps to subclasses.
When to use	Invariant parts of an algorithm should be implemented once; subclasses customise specific steps.
Trade-offs	Tight coupling between base and subclass; Liskov substitution must be respected.

Java Example

```

abstract class DataProcessor {
    // Template method — final to prevent override
    public final void process() {
        readData();
        processData();
        writeData();
    }
    protected abstract void readData();
    protected abstract void processData();
    protected void writeData() {
        System.out.println("Writing results to output");
    }
}

class CsvProcessor extends DataProcessor {
    protected void readData() { System.out.println("Reading CSV file"); }
    protected void processData() { System.out.println("Parsing CSV rows"); }
}

class JsonProcessor extends DataProcessor {
    protected void readData() { System.out.println("Reading JSON file"); }
}

```

```

protected void processData() { System.out.println("Parsing JSON
nodes"); }

}

// Usage

new CsvProcessor().process();

new JsonProcessor().process();

```

4.5 State

Attribute	Detail
Intent	Allow an object to alter its behaviour when its internal state changes.
When to use	Order workflows, vending machines, traffic lights — anything with well-defined state machines.
Trade-offs	Can increase number of classes; state transitions must be carefully managed.

Java Example

```

interface TrafficLightState {
    void handle(TrafficLight ctx);

    String name();
}

class RedState implements TrafficLightState {
    public void handle(TrafficLight ctx) { ctx.setState(new Green-
State()); }

    public String name() { return "RED"; }
}

class GreenState implements TrafficLightState {
    public void handle(TrafficLight ctx) { ctx.setState(new Yellow-
State()); }

    public String name() { return "GREEN"; }
}

class YellowState implements TrafficLightState {
    public void handle(TrafficLight ctx) { ctx.setState(new RedState());
}

    public String name() { return "YELLOW"; }
}

class TrafficLight {

```

```

private TrafficLightState state = new RedState();

public void setState(TrafficLightState s) { this.state = s; }

public void change() {
    System.out.println("Current: " + state.name());
    state.handle(this);
}

}

// Usage

TrafficLight light = new TrafficLight();

light.change(); // RED -> GREEN

light.change(); // GREEN -> YELLOW

light.change(); // YELLOW -> RED

```

5. Quick-Reference Summary

The table below provides a bird's-eye view of all patterns covered in this document.

Pattern	Category	One-line intent
Singleton	Creational	One instance, global access point
Factory Method	Creational	Let subclasses decide which class to instantiate
Builder	Creational	Fluent step-by-step construction of complex objects
Prototype	Creational	Clone existing objects instead of new construction
Adapter	Structural	Translate one interface to another
Decorator	Structural	Add behaviour dynamically without subclassing
Facade	Structural	Simplified interface to a complex subsystem
Composite	Structural	Tree structures for part-whole hierarchies
Observer	Behavioural	Notify all dependents on state change
Strategy	Behavioural	Swap algorithms at runtime
Command	Behavioural	Encapsulate requests as objects (undo/redo)
Template Method	Behavioural	Skeleton algorithm; subclasses fill in steps
State	Behavioural	Change behaviour as internal state changes

6. Further Reading

Design Patterns: Elements of Reusable Object-Oriented Software — Gamma, Helm, Johnson, Vlissides (1994)

Head First Design Patterns — Freeman, Robson (2004, updated 2021)

Refactoring.guru — <https://refactoring.guru/design-patterns> — free online catalogue with animated diagrams

Effective Java (3rd ed.) — Joshua Bloch (2018) — practical pattern usage in the JDK